



FIELDNOTE

An offline-first field inspection app

Final project documentation

Student	Muhammad Ibrahiem
Internship ID	ZYNVEX-CERT-1271
Program	Zynvex Solutions internship, Batch 3 (four phases, 24 Aug – 20 Sep 2026)
Repository	https://github.com/Ibrahiem17/fieldnote
Version	1.1.0 (Android production build)
Stack	React Native (Expo SDK 57), TypeScript, SQLite, Supabase

1. What Fieldnote is (in simple words)

In one line: Fieldnote is a phone app that replaces the paper checklist a person carries when they check a place — and it works even when there is no internet.

The problem. When someone inspects a place — a roof, a machine, a building site — they walk around and check things one by one. Usually they write on paper, take photos on their phone, and then spend time typing it all up later. Papers get lost, photos get mixed up, and many sites have no phone signal at all, so most apps stop working there.

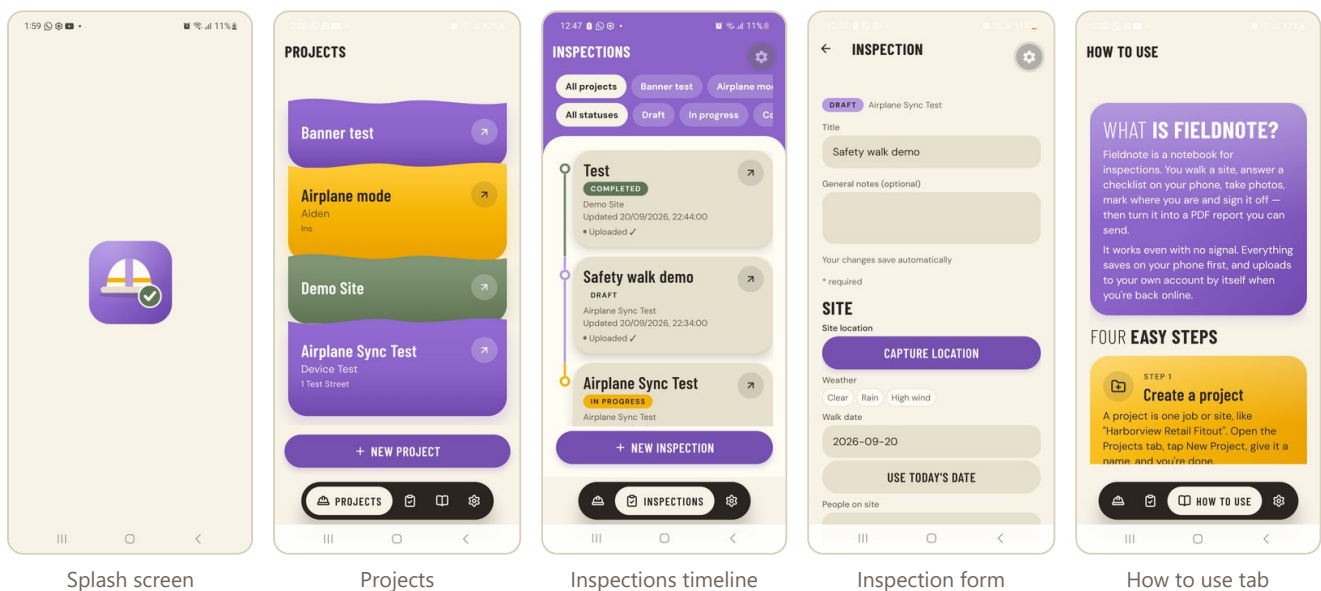
What the app does. With Fieldnote the person does all of it on their phone:

- choose a checklist (for example "Roof Inspection") and answer the questions;
- take photos, mark where they are, and sign with their finger;
- press one button to make a **PDF report** they can send to anyone.

Why it works without internet. Everything is saved on the phone first, so nothing depends on a signal. When the phone finds internet again, the app quietly sends the work to the person's own online account. If the internet drops halfway, nothing is lost — the app just tries again later.

An example. Sara visits a roof in a place with no signal. She opens Fieldnote, starts a "Roof Inspection", answers the questions, takes three photos and signs. Later, back in the car with signal, her work uploads by itself, and she creates a PDF report and sends it to her client.

Who it is for. Anyone who checks places or equipment and has to write a report: site supervisors, safety officers, maintenance teams. Each person only sees their own work.



Screenshots taken on a Samsung Galaxy A51 running the v1.1.0 production build.

2. Features

- **Projects and inspections** — create, edit and delete; filter inspections by project and status; swipe to delete.
- **Template-driven forms** — a template is plain data; ten field types: text, long text, number, select, multi-select, yes/no, date, photo, GPS location and signature. Fields can appear only when another

answer calls for them (conditional visibility). Required questions are marked and checked before an inspection can be completed.

- **Photos, GPS and signature** — camera capture (resized on the phone), a location stamp with a clear failure path, and a finger-drawn signature.
- **Automatic upload** — a hand-written sync engine with a queue, retries and conflict handling; a strip at the top says in plain words whether changes are waiting or uploaded.
- **PDF report** — generated on the phone with no network, shared through the phone's share menu.
- **Sign-in** — email and password through Supabase Auth; each user only ever sees their own data.
- **Built-in help** — a "How to use" tab that opens once on first launch and explains the app in plain language.
- **Polish** — a colour-coded design system, animated cards and tabs (respecting the phone's reduce-motion setting), accessibility labels, loading and error states on every screen.

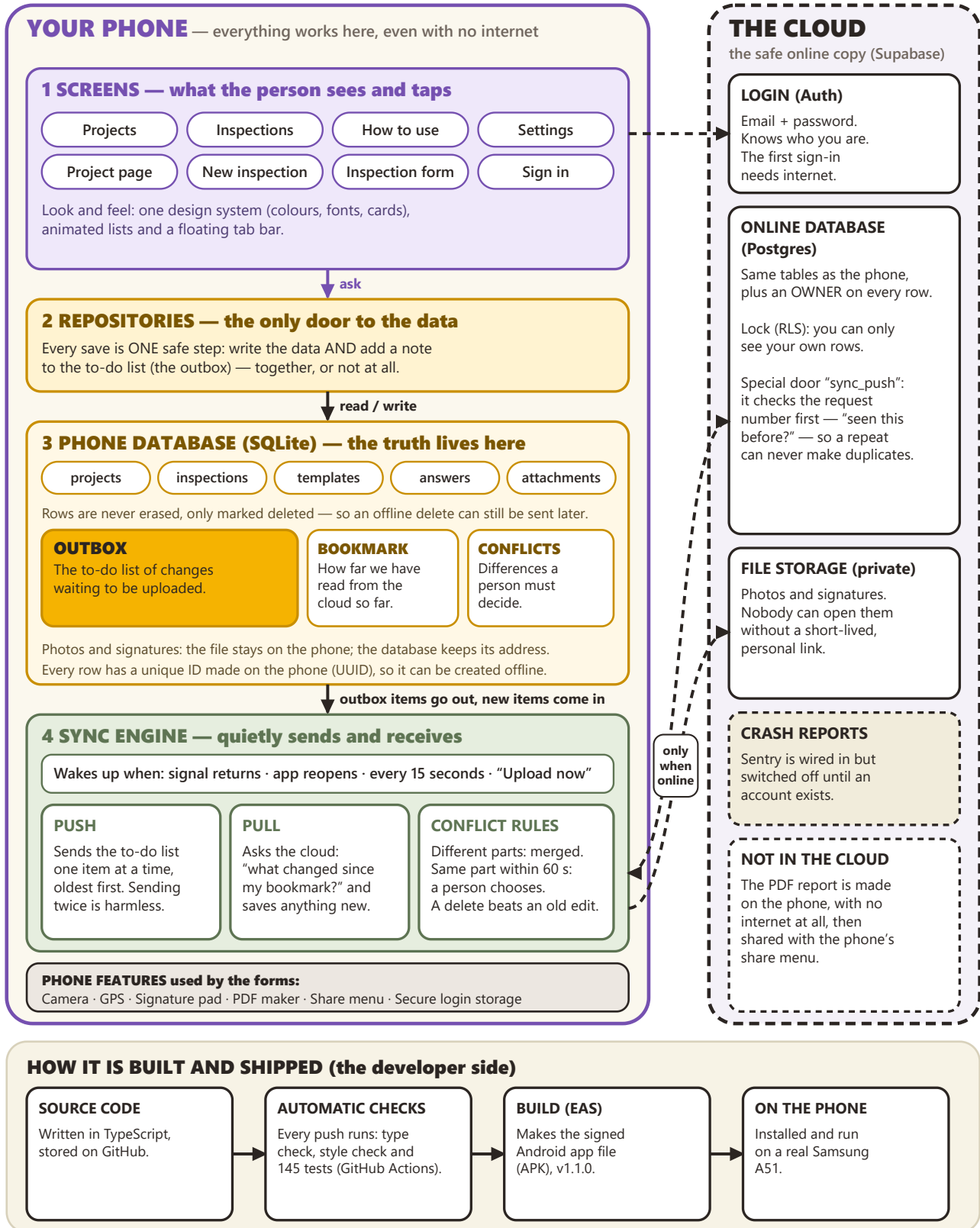
3. How to use it

Step	What the user does
1. Create a project	Projects tab → New Project → give the job a name.
2. Start an inspection	Open the project → New Inspection → choose the type and a title.
3. Fill it in	Answer the questions (they save automatically); add photos, capture location, sign on screen.
4. Finish and share	Mark as complete → Create PDF report → share or save.

With no signal, work continues normally; a strip at the top shows how many changes are waiting, and they upload by themselves when signal returns.

4. Architecture — the big picture

How to read this diagram: the left side is everything that lives on the phone, the right side is the online copy in the cloud. The dashed lines are the internet, which is used only when it is available. The bottom strip shows how the app is built and shipped.



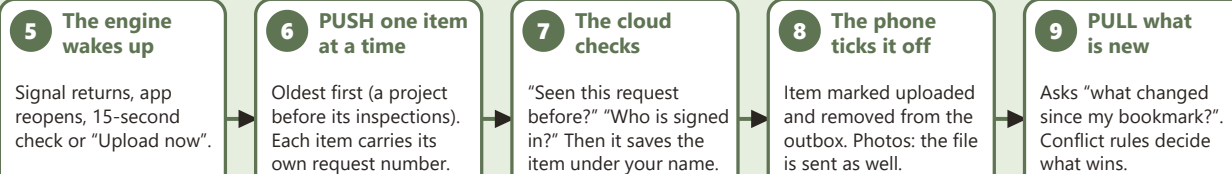
4b. What happens when you save something

Follow the numbers 1 to 9. Steps 1–4 always work; steps 5–9 happen by themselves when there is internet.

A • SAVING ON THE PHONE — always works, even with no signal



B • SENDING TO THE CLOUD — only when internet is there



WHAT IF SOMETHING GOES WRONG?

No signal

Step 6 just waits. Nothing is lost; the to-do list keeps growing safely.

App closed halfway

The item stays in the outbox and is tried again. Sending twice is harmless.

Sent twice by mistake

The cloud recognises the request number and ignores the second copy.

Refused again and again

It is set aside ("dead-letter"). Only the Retry button tries it again.

Words used in the diagrams

Word	What it means
Outbox	A to-do list on the phone of every change that still has to be uploaded.
Sync	Quietly matching the phone and the cloud so both end up with the same information.
Push / Pull	Push = send my changes up. Pull = fetch other changes down.
Bookmark (cursor)	A note of how far the phone has already read from the cloud.
Safe to repeat (idempotent)	Sending the same request twice has the same result as sending it once — so retries never create duplicates.
Conflict	The same thing was changed in two places. Different parts are merged; the same part is decided by a person.
RLS (row-level security)	A lock in the online database: each signed-in person can only see and change their own rows.
Soft delete	A deleted item is only marked as deleted, so the delete can still be sent to the cloud.
UUID	A long unique ID made on the phone, so nothing needs the internet to get an ID.
Dead-letter	An item the cloud keeps refusing; it is set aside until a person taps Retry.

The building rules behind the diagrams

- **Screens never touch the database.** They call repository functions; only repositories use Drizzle/SQL.
- **Every change writes an outbox entry in the same transaction** as the data change, so "what needs uploading" can never disagree with "what changed".
- **Soft delete only** — a `deleted_at` timestamp, so an offline delete can still be sent to the server later.
- **Device-generated UUIDs** for every row, so records can be created with no server reachable.
- **Timestamps** are epoch milliseconds on the phone; the server's own timestamp (`server_updated_at` , set by a database trigger) is the only clock ever used to decide a conflict.
- **One door per service** — a single Supabase client and a single Sentry module.

Layer	Technology
App framework	React Native with Expo SDK 57, TypeScript (strict), Expo Router (file-based navigation)
Local database	SQLite (expo-sqlite) with Drizzle ORM and generated migrations
Backend	Supabase: Postgres, Auth, Row Level Security, private file Storage
UI	FlashList, Reanimated, Gesture Handler, SVG, linear gradients, Lucide icons, Barlow Condensed + DM Sans fonts
Reports	expo-print (HTML → PDF) and expo-sharing
Quality	Jest (145 tests), ESLint, TypeScript, GitHub Actions CI, Sentry (wired, not activated)
Build	EAS Build (signed Android APK)

5. Data model

On the phone: `projects`, `templates`, `inspections`, `answers` (one row per answered field), `attachments` (photos and signatures), `outbox` (the upload queue), `sync_state` (the pull cursor) and `conflicts` (differences flagged for a person to decide). Every entity table carries `id`, `created_at`, `updated_at`, `deleted_at` and `sync_status`. The server mirrors these tables and adds `owner_id` and `server_updated_at`, plus a `sync_idempotency_keys` table.

6. The sync engine

- **Push** goes through one database function (`sync_push`) that checks an **idempotency key** first (the outbox row's own id), so a retried upload can never create a duplicate. Rows are pushed one at a time, parents before children.
- **Retries** use exponential backoff with jitter (1 s doubling to a 5-minute cap). A row that keeps being rejected is **dead-lettered** and only retried by an explicit "retry failed" action. A push that never reached the server (no signal) does *not* spend a retry attempt.
- **Pull** reads changes since a cursor (the server's own timestamp string). A pulled row that has a pending local edit is passed through the conflict policy instead of being blindly overwritten.
- **Conflict policy** (a pure, unit-tested function): different fields merge silently; the same field edited within 60 seconds on both sides is flagged for a person to choose; a delete beats an older edit.
- **Photos** upload as three idempotent steps (row → file → link the file path), and the queue entry is only cleared after all three succeed, so a killed app retries cleanly.
- **Sync order** is always push then pull, through one entry point, triggered on reconnect, on returning to the app, periodically while online, and by an "Upload now" button.

7. Security and privacy

- Every server table has Row Level Security: a user can only read and write rows where `owner_id` equals their own account. This was tested by trying to break it with two real accounts.
- `owner_id` is derived from the signed-in session on the server, never trusted from the request.
- Photos live in a **private** storage bucket; the database stores only the file path, and viewing a photo requests a short-lived signed link.
- The app contains only the public (anon) Supabase key. The service-role key and database password are never in the app or the repository; the `.env` file is git-ignored.
- Sessions are stored in the phone's encrypted storage (SecureStore).

8. Design system and user experience

The interface follows one design system (`src/theme/design.ts`): warm cream backgrounds, purple / amber / olive colour-coded cards with rippled tops, a purple timeline sheet for inspections, a floating dark navigation pill, large rounded shapes, tinted shadows and small spring animations. Small text uses deeper colour variants so contrast stays at or above 4.5:1; a test fails if that changes. Capital letters are used only for fixed headings, never for text a person typed. The PDF report deliberately keeps its own, separate look. New users are helped by a plain-language **How to use** tab, required-question markers, a "Use today's date" button, friendly error messages, and an app icon and splash screen drawn for the project (a hard hat

with an inspection check).

9. What was delivered, phase by phase

Module / phase	Scope	Outcome
1 — Foundation	Navigation, design tokens, offline SQLite data layer, repositories, seed data	Complete; documented in TEST-RESULTS-PHASE-1
2 — Forms and media	Dynamic form engine, all ten field types, camera, GPS, signature, validation	Complete; a third template was added with no engine change
3 — Sync and auth	Supabase auth, RLS, outbox sync, conflict resolution, background photo upload	Complete; 15 checks run live against a real Supabase project
4 — Reporting and release	PDF reports, sharing, deep links, animation, accessibility, automated tests, CI, production build	Complete, with honest labels for what needed a device
After phase 4	Real-phone testing and fixes, usability pass, full visual redesign, icon, splash, in-app guide, v1.1.0 APK	Done and checked on a Samsung A51

10. Testing and verification

Automated

- **145 unit and integration tests** in 22 suites: sync engine, conflict resolution, backoff, validation, visibility rules, template ids and migrations, report HTML, guide wording, brand-asset sizes, and repository tests against a real in-memory SQLite database.
- **TypeScript strict**: 0 errors. **ESLint**: clean.
- **Continuous integration** on GitHub Actions runs typecheck, lint and tests on every push. It was verified by deliberately pushing a type error and watching the run fail, then recover.

On a real phone (Samsung Galaxy A51, Android 13)

Verified on a real phone
Install and launch of the standalone production build; session and data survive a restart; camera capture; a real GPS fix and the location-denied path; signature capture; creating projects and inspections through the real screens; the full Site Safety Walk; offline create → reconnect → automatic upload; the upload strip; the PDF being generated and the share menu opening; splash screen; one-time guide on first launch; the redesigned screens. Also checked by hand by the developer on the phone: what the finished PDF report looks like when opened, a deep link with the app fully closed, how the animations feel, the launcher (and themed) icon, and the sign-in screen after the redesign.

The real-phone testing found and fixed many bugs the web preview and unit tests could not see (for example: a "Select" field that was free text, fake GPS coordinates saved when location failed, a placeholder photo saved on cancel, and edits made before an upload being silently lost). Each is recorded with its reasoning in docs/DESIGN.md.

11. Build, install and release

- Build: `eas build --profile production --platform android` produces a signed APK (version 1.1.0). It was installed on the phone with the development server switched off and ran correctly.
- Run from source: `npm install`, copy `.env.example` to `.env` with your own Supabase URL and anon key, then `npm start`. Checks: `npm run typecheck`, `npm run lint`, `npm test`.
- Not on Google Play or the App Store: both need paid developer accounts. `docs/RELEASE.md` lists exactly what remains.

12. Known limitations and future work

- A photo taken on one phone cannot yet be displayed on a second phone after syncing (the file is stored; the app only shows local files).
- Dates are typed (with a "Use today's date" button) rather than chosen from a calendar, which needs a new native module.
- Crash reporting (Sentry) is installed but inactive until a Sentry project is created.
- About 17 Expo packages are a few patch versions behind; upgrading is planned as its own step with a full re-test.
- Light mode only; iOS untested; performance and screen-reader behaviour not yet measured on a device.

13. Repository and documents

GitHub: <https://github.com/lbrahiem17/fieldnote> — the README covers setup, scripts, structure, architecture and known issues. Supporting documents in `docs/`: `SYNC.md` (the sync protocol), `DESIGN.md` (every decision with the alternatives rejected, D-001 to D-047), `UNDERSTANDING.md` (the app in plain words), `BABY.md` (line-by-line code explanations), `RELEASE.md`, `TEST-RESULTS-PHASE-1...4.md` and `TEST-RESULTS-DEVICE.md`.